



Nivel 1

1.

- Importa como un DataFrame el archivo sprint10.xlsx. Asegúrate de que el archivo se importa correctamente, con los nombres de columnas que le corresponden, sin manipular el archivo original.
- Ordena el DataFrame por el país de origen. En caso de empate, ordena por el nombre de la ciudad.
- Muestra las primeras 10 filas.

Adicionalmente, realiza un *print* donde compruebe que el DNI sólo tiene valores únicos.

```
import pandas as pd

# leer archivo y colocar la fila 4 como encabezado
df = pd.read_excel(r"C:\Users\gabriel\Desktop\Especializacion\python\SPRINT 10\sprint10.xlsx", header=3)

# cambiar la columna unnamed por ID, ya que así esta en el archivo original

df = df.rename(columns={"Unnamed: 0": "ID",
                        "País d'origen": "País_de_origen",
                        "Dia de Naixement": "Dia_de_Naixement",
                        "Mes de Naixement": "Mes_de_Naixement",
                        "Any de Naixement": "Any_de_Naixement",
                        "Salari mensual": "Salari_mensual",
                        "No Fills": "No_Fills",
                        "Grup Professional": "Grup_Professional",
                        "Gènere": "Gènere"
                       })

# ordenar valores por país de origen y ciudad

df = df.sort_values(by=["País_de_origen", "Ciutat"], ascending=[True, True])

# mostrar si el Dni se repite o no

print("El Dni no se repite:", df['DNI'].is_unique)

# mostrar las primeras 10 filas del dataframe

df.head(10)
```

[3] ✓ 32s
... El Dni no se repite: True

	ID	Nom	Cognoms	DNI	País_de_origen	Ciutat	Dia_de_Naixement	Mes_de_Naixement	Any_de_Naixement	Genere	Salari_mensual	Fills	No_Fills	Grup_Professional
21	21	Mia	Schneider Fischer	28973553Z	Alemanya	Berlín	22	10	1976	A	951 €	NaN	1.0	Grup A
154	154	Laura	Schneider Fischer	37399141L	Alemanya	Berlín	2	2	1958	D	1.769 €	1.0	NaN	Grup B
224	224	Lea	Schneider Schneider	37368317L	Alemanya	Berlín	23	10	2005	D	2.013 €	NaN	1.0	Grup B

2.

- Crea una columna que sea el nombre completo.
- Crea una columna si la persona ha nacido en España o no.
- Pone el DNI como índice del DataFrame (nombres de filas).
- Sustituye el nombre de las columnas Día de Nacimiento, Mes de Nacimiento y Año de Nacimiento por Día, Mes y Año.
- Sustituye a H por Hombre, D por Mujer, A por Otros y NC por un dato faltante (enano/null/na).

Muestra todos los cambios que has realizado en una sola tabla.

```
import numpy as np

#insertar en la posicion 3 la columna con el nombre completo, concatenando el nombre y el apellido
df.insert(3, "Nom_Complet", df["Nom"] + " " + df["Cognoms"], True)

# insertar columna que diga si ha nacido en españa o no, con el valor True o False
df.insert(7, "Nascuda_a_Espanya", df["Pais_de_origen"] == "Espanya", True)

#hacer que el index sea iguala al DNI
df.index = df["DNI"]
df.drop(columns=["DNI"], inplace=True)

#cambiar nombre de las columnas
df = df.rename(columns={"Dia_de_Naixement": "Dia",
                        "Mes_de_Naixement": "Mes",
                        "Any_de_Naixement": "Any"})

#cambiar los valores de la columna género, poniendo en lugar de A, D, H y NC, Altres, Dona, Home y NaN respectivamente
df["Genere"] = df["Genere"].replace(
    {
        "A": "Altres",
        "D": "Dona",
        "H": "Home",
        "NC": np.nan
    })

cambiados = df[["Nom_Complet", "Nascuda_a_Espanya", "Dia", "Mes", "Any", "Genere"]]
cambiados
```

[4] ✓ 0.0s

	Nom_Complet	Nascuda_a_Espanya	Dia	Mes	Any	Genere
DNI						
28973553Z	Mia Schneider Fischer	False	22	10	1976	Altres
37399141L	Laura Schneider Fischer	False	2	2	1958	Dona
37368317L	Lea Schneider Schneider	False	23	10	2005	Dona
21390098Z	Mia Fischer	False	11	8	1950	Dona

3.

- Junta las columnas Hijos y No Hijos en una sola columna, utilizando el método .apply() y definiendo una función que resuelva el problema. La columna nueva debe llamarse "Hijos" y tomar los valores "Sí" o "No".

```
#Uso apply para modificar la columna fills, dependiendo si tiene hijos o no, y elimino la fila no fills del df
df["Fills"] = df["Fills"].apply(lambda x: "Si" if x == 1.0 else "No")

df = df.drop(columns=["No_Fills"])

df
```

[5] ✓ 0.0s

	ID	Nom	Cognoms	Nom_Complet	Pais_de_origen	Ciutat	Nascuda_a_Espanya	Dia	Mes	Any	Genere	Salari_mensual	Fills	Grup_Professional
DNI														
28973553Z	21	Mia	Schneider Fischer	Mia Schneider Fischer	Alemanya	Berlin	False	22	10	1976	Altres	951 €	No	Grup A
37399141L	154	Laura	Schneider Fischer	Laura Schneider Fischer	Alemanya	Berlin	False	2	2	1958	Dona	1.769 €	Si	Grup B
37368317L	224	Lea	Schneider Schneider	Lea Schneider Schneider	Alemanya	Berlin	False	23	10	2005	Dona	2.013 €	No	Grup B
21390098Z	278	Mia	Fischer	Mia Fischer	Alemanya	Berlin	False	11	8	1950	Dona	1.557 €	Si	Grup B
44060014R	602	Jonas	Schneider	Jonas Schneider	Alemanya	Berlin	False	22	11	1985	Home	2.754 €	Si	Grup D
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
89577876S	547	Emily	Taylor Jones	Emily Taylor Jones	Regne Unit	Manchester	False	28	3	1958	Dona	2.033 €	No	Grup B
57441590Y	728	George	Brown Jones	George Brown Jones	Regne Unit	Manchester	False	27	12	1979	Home	1.130 €	Si	Grup A
58204038A	751	Olivia	Brown Brown	Olivia Brown Brown	Regne Unit	Manchester	False	28	8	1952	Altres	1.023 €	No	Grup A
28367234K	854	Isla	Jones Brown	Isla Jones Brown	Regne Unit	Manchester	False	28	3	1999	Dona	1.197 €	No	Grup A
82046699J	872	William	Smith	William Smith	Regne Unit	Manchester	False	19	5	1997	Home	1.405 €	Si	Grup A

4.

- Crea una tabla resumen que permita ver el sueldo medio, medio, mínimo y máximo por Género.
- Ordena la tabla en función del sueldo medio.

F

lo primero es convertir la columna de salario mensual a tipo numérico, ya que actualmente es un objeto (string) y no se pueden hacer operaciones matemáticas con ella
se debería haber hecho al mismo momento que la nueva tabla??
df["Salari_mensual"] =(
 df["Salari_mensual"]
 .str.replace(" €", "", regex=False)
 .str.replace(".", "", regex=False)
 .astype(int)
)

df

[6] ✓ 0.0s

...

	ID	Nom	Cognoms	Nom_Complet	Pais_de_origen	Ciutat	Nascuda_a_Espanya	Dia	Mes	Any	Genere	Salari_mensual	Fillis	Grup_Professional
DNI														
28973553Z	21	Mia	Schneider Fischer	Mia Schneider Fischer	Alemanya	Berlin	False	22	10	1976	Altres	951	No	Grup A
37399141L	154	Laura	Schneider Fischer	Laura Schneider Fischer	Alemanya	Berlin	False	2	2	1958	Dona	1769	Si	Grup B
37368317L	224	Lea	Schneider Schneider	Lea Schneider Schneider	Alemanya	Berlin	False	23	10	2005	Dona	2013	No	Grup B
21390098Z	278	Mia	Fischer	Mia Fischer	Alemanya	Berlin	False	11	8	1950	Dona	1557	Si	Grup B
44060014R	602	Jonas	Schneider	Jonas Schneider	Alemanya	Berlin	False	22	11	1985	Home	2754	Si	Grup D
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
89577876S	547	Emily	Taylor Jones	Emily Taylor Jones	Regne Unit	Manchester	False	28	3	1958	Dona	2033	No	Grup B
57441590Y	728	George	Brown Jones	George Brown Jones	Regne Unit	Manchester	False	27	12	1979	Home	1130	Si	Grup A
58204038A	751	Olivia	Brown Brown	Olivia Brown Brown	Regne Unit	Manchester	False	28	8	1952	Altres	1023	No	Grup A
28367234K	854	Isla	Jones Brown	Isla Jones Brown	Regne Unit	Manchester	False	28	3	1999	Dona	1197	No	Grup A
82046699J	872	William	Smith	William Smith	Regne Unit	Manchester	False	19	5	1997	Home	1405	Si	Grup A

```
#nueva tabla con el group by genero y todos los calculos
nueva_tabla = (
    df.groupby("Genero", as_index=True)
        .agg(
            Cantidad=("Genero", "count"),
            Salari_Mitja =("Salari_mensual", "mean"),
            Salari_Mediana=("Salari_mensual", "median"),
            Salari_Maxim=("Salari_mensual", "max"),
            Salari_Minim=("Salari_mensual", "min")
        ).round(2)
    ).sort_values(by="Salari_Mitja", ascending=False)

nueva_tabla
```

[7] ✓ 0.0s

	Cantidad	Salari_Mitja	Salari_Mediana	Salari_Maxim	Salari_Minim
Genero					
Home	449	1643.25	1531.0	3356	737
Altres	51	1626.59	1545.0	3175	703
Dona	440	1469.44	1361.5	3021	665

5.

- Crea una tabla resumen con el salario medio por género (filas) y país de origen (columnas).
- Añade las medias a los márgenes de la mesa.

(EXTRA): Aplica formato condicional en la tabla para ver en un color más intenso los valores más elevados

```

import pandas as pd

# Crear tabla resumen (pivot)

tabla_resumen = pd.pivot_table(
    df,
    values="Salari_mensual",
    index="Genere" ,
    columns="Pais_de_origen",
    aggfunc="mean",
    margins=True,
    margins_name="Media_Total"
)

# Redondear a 2 decimales

tabla_resumen = tabla_resumen.round(2)

# Mostrar con formato moneda + color

tabla_resumen = (
    tabla_resumen
    .style
    .format("{:,.2f} €")
    .background_gradient(cmap="YlGnBu")
)

tabla_resumen

```

✓ 1.5s

Pais_de_origen	Alemanya	Argentina	Colòmbia	Espanya	França	Itàlia	Marroc	Mèxic	Portugal	Regne Unit	Media_Total
Genere											
Altres	951.00 €	1,141.00 €	1,030.00 €	1,706.18 €	nan €	1,423.00 €	1,365.00 €	1,372.00 €	1,765.00 €	1,921.00 €	1,626.59 €
Dona	1,804.31 €	1,291.80 €	1,497.75 €	1,460.16 €	1,566.47 €	1,247.18 €	1,405.21 €	1,517.80 €	1,488.55 €	1,489.46 €	1,469.44 €
Home	2,067.43 €	1,583.29 €	1,554.67 €	1,682.11 €	1,389.25 €	1,672.88 €	1,531.00 €	1,625.00 €	1,497.00 €	1,162.56 €	1,643.25 €
Media_Total	1,851.38 €	1,463.39 €	1,495.54 €	1,581.21 €	1,462.73 €	1,425.95 €	1,447.33 €	1,558.42 €	1,523.33 €	1,423.56 €	1,560.99 €

6.

- Crea una nueva columna que sea la fecha de nacimiento en formato Datetime a partir de las columnas día, mes y año. Utilizando esta columna crea una función que dada una fecha, te calcule la edad actual a día de hoy.
- Utiliza la función que acabas de crear para generar una columna nueva en DataFrame con la edad actual.

```
import pandas as pd
from datetime import datetime

# funcion para crear columna con fecha de nacimiento por cada fila
def crear_fecha(fila):
    try:
        return datetime(int(fila["Any"]), int(fila["Mes"]), int(fila["Dia"]))
    except:
        return pd.NaT
# creo la columna con la fecha de nacimiento utilizando la funcion creada anteriormente
df["Data_Naixement"] = df.apply(crear_fecha, axis=1)

# funcion para calcular la edad a partir de la fecha de nacimiento
def calcular_edad(fecha_nacimiento):

    if pd.isna(fecha_nacimiento):
        return None

    hoy = datetime.now()
    edad = hoy.year - fecha_nacimiento.year

    # Ajuste si aún no ha cumplido años este año, si el mes y el día de hoy son menores que el mes y el día de nacimiento
    if (hoy.month, hoy.day) < (fecha_nacimiento.month, fecha_nacimiento.day):
        edad -= 1

    return edad
# creo la columna con la edad actual utilizando la funcion creada anteriormente
df["Edat_actual"] = df["Data_Naixement"].apply(calcular_edad)

df
```

✓ 0.0s

✓ 0.0s Python

	ID	Nom	Cognoms	Nom_Complet	País_de_origen	Ciutat	Nascuda_a_Espanya	Dia	Mes	Any	Genere	Salari_mensual	Fillis	Grup_Professional	Data_Naixement	Edat_actual
DNI																
28973553Z	21	Mia	Schneider Fischer	Mia Schneider Fischer	Alemanya	Berlín	False	22	10	1976	Altres	951	No	Grup A	1976-10-22	49
37399141L	154	Laura	Schneider Fischer	Laura Schneider Fischer	Alemanya	Berlín	False	2	2	1958	Dona	1769	Si	Grup B	1958-02-02	68
37368317L	224	Lea	Schneider Schneider	Lea Schneider Schneider	Alemanya	Berlín	False	23	10	2005	Dona	2013	No	Grup B	2005-10-23	20
21390098Z	278	Mia	Fischer	Mia Fischer	Alemanya	Berlín	False	11	8	1950	Dona	1557	Si	Grup B	1950-08-11	75
44060014R	602	Jonas	Schneider	Jonas Schneider	Alemanya	Berlín	False	22	11	1985	Home	2754	Si	Grup D	1985-11-22	40
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
89577876S	547	Emily	Taylor Jones	Emily Taylor Jones	Regne Unit	Manchester	False	28	3	1958	Dona	2033	No	Grup B	1958-03-28	67
57441590Y	728	George	Brown Jones	George Brown Jones	Regne Unit	Manchester	False	27	12	1979	Home	1130	Si	Grup A	1979-12-27	46
58204038A	751	Olivia	Brown Brown	Olivia Brown Brown	Regne Unit	Manchester	False	28	8	1952	Altres	1023	No	Grup A	1952-08-28	73
28367234K	854	Isla	Jones Brown	Isla Jones Brown	Regne Unit	Manchester	False	28	3	1999	Dona	1197	No	Grup A	1999-03-28	26
82046699J	872	William	Smith	William Smith	Regne Unit	Manchester	False	19	5	1997	Home	1405	Si	Grup A	1997-05-19	28

1000 rows × 16 columns

🔗 Generate

+ Code

+ Markdown



Nivel 2

1.

- Utilizando el siguiente DataFrame, adjunta la columna "Incremento" al dataframe del nivel anterior.
- Actualiza la columna salario en función de los porcentajes que se adjuntan. No modifiques manualmente los incrementos, escribe código Python para realizar las conversiones necesarias.

```
df_increment = pd.DataFrame({"Grupo":["Grupo A","Grupo B","Grupo C", "Grupo D"], "Incremento":  
["5%","3,5%","2%","8%"]})
```

	Grup	Increment
0	Grup A	5%
1	Grup B	3,5%
2	Grup C	2%
3	Grup D	8%

```
#crei el dataframe con los porcentajes de incremento para cada grupo profesional  
df_increment = pd.DataFrame({"Grup":["Grup A","Grup B","Grup C", "Grup D"], "Incremento":  
["5%","3,5%","2%","8%"]})
```

```
df_increment
```

✓ 0.0s

	Grup	Increment
0	Grup A	5%
1	Grup B	3,5%
2	Grup C	2%
3	Grup D	8%

```
# hago .map que es como un for, donde por cada valor de Grup Profesional, busca el valor correspondiente y le asigna el valor de incremento a la nueva columna Incremento  
df["Incremento"] = df["Grup_Profesional"].map(  
    df_increment.set_index("Grup")["Incremento"] #para eso lo paso a serie panda, es como un diccionario. clave valor. Grup como clave index e increment como unica columna valor  
)  
  
df["Salari_mensual"] = (  
    df["Salari_mensual"] * (  
        1 + df["Incremento"]  
        .str.replace("%", "", regex=False)  
        .str.replace(",", ".", regex=False)  
        .astype(float)  
        / 100  
    )  
) .round(2)
```

```
df
```

✓ 0.0s

Python

Python																	
	ID	Nom	Cognoms	Nom_Complet	Pais_de_origen	Ciutat	Nascuda_a_Espanya	Dia	Mes	Any	Genere	Salari_mensual	Fills	Grup_Professional	Data_Naixement	Edat_actual	Increment
DNI																	
28973553Z	21	Mia	Schneider Fischer	Mia Schneider Fischer	Alemanya	Berlín	False	22	10	1976	Altres	998.55	No	Grup A	1976-10-22	49	5
37399141L	154	Laura	Schneider Fischer	Laura Schneider Fischer	Alemanya	Berlín	False	2	2	1958	Dona	1830.92	Si	Grup B	1958-02-02	68	3,5
37368317L	224	Lea	Schneider Schneider	Lea Schneider Schneider	Alemanya	Berlín	False	23	10	2005	Dona	2083.46	No	Grup B	2005-10-23	20	3,5
21390098Z	278	Mia	Fischer	Mia Fischer	Alemanya	Berlín	False	11	8	1950	Dona	1611.50	Si	Grup B	1950-08-11	75	3,5
44060014R	602	Jonas	Schneider	Jonas Schneider	Alemanya	Berlín	False	22	11	1985	Home	2974.32	Si	Grup D	1985-11-22	40	5
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
89577876S	547	Emily	Taylor Jones	Emily Taylor Jones	Regne Unit	Manchester	False	28	3	1958	Dona	2104.15	No	Grup B	1958-03-28	67	3,5

2.

Utilizando un bucle, exporta en 4 archivos (formato .xlsx o .csv) los datos de cada Grupo Profesional.

Por ejemplo: "datos_GrupA.xlsx" , "datos_GrupB.xlsx" ...

Exporta un 5º DataFrame en formato .xlsx o .csv que contenga cuántos trabajadores hay por cada Grupo Profesional, cuál es su sueldo medio y cuál es su edad media.

```
# crear bucle, por cada grup, crear un archivo dades_grup.xlsx con las filas correspondientes a ese grup
for grup, dades in df.groupby("Grup_Professional"):

    # Limpiar nombre para archivo (sin espacios)
    nom_fitxer = f"dades_{grup.replace(' ', '')}.xlsx"

    dades.to_excel(nom_fitxer)

df_resum = (
    df
    .groupby("Grup_Professional")
    .agg(
        treballadors=("Grup_Professional", "count"),
        salari_mig=("Salari_mensual", "mean"),
        edat_mediana=("Edat_actual", "median")
    )
    .reset_index()
)
df_resum["salari_mig"] = df_resum["salari_mig"].round(2)
df_resum.to_excel("resum_grups.xlsx", index=False)
```

[15] ✓ 1.3s

Escritorio > Especializacion > python > SPRINT 10 >				
Ordenar Ver				
Nombre	Fecha de modificación	Tipo	Tamaño	
graficos	09/03/2026 11:13	Carpeta de archivos		
graficos_iris	03/03/2026 12:38	Carpeta de archivos		
graficos_titanic	03/03/2026 12:38	Carpeta de archivos		
dades_GrupA.xlsx	09/03/2026 11:09	Archivo XLSX	52 KB	
dades_GrupB.xlsx	09/03/2026 11:09	Archivo XLSX	36 KB	
dades_GrupC.xlsx	09/03/2026 11:09	Archivo XLSX	19 KB	
dades_GrupD.xlsx	09/03/2026 11:09	Archivo XLSX	11 KB	
matriu_distancies.xlsx	25/02/2026 10:21	Archivo XLSX	18 KB	
resum_grups.xlsx	09/03/2026 11:09	Archivo XLSX	5 KB	
SPRINT 10.ipynb	09/03/2026 11:37	Archivo de origen ...	99 KB	
sprint10.xlsx	25/02/2026 10:21	Archivo XLSX	93 KB	
Untitled-1.ipynb	03/03/2026 15:01	Archivo de origen ...	106 KB	



Nivel 3

El nivel 3 de este sprint es totalmente diferente a otros sprints que has hecho hasta ahora, ya que son ejercicios más abstractos que requieren pelearse bastante. No siguen con el mismo dataset de los niveles anteriores, sino que te plantean dos situaciones nuevas totalmente distintas entre sí.

1.

Crea una función que tome un dataframe como parámetro de entrada.

La función debe crear (y exportar) un gráfico automáticamente para cada columna del dataframe. Por ejemplo:

-
- un histograma/boxplot si la variable es numérica
 - unas barras de los valores más frecuentes si es categórica
 - unas barras de los años más frecuentes si el dato está en formato fecha.
-

La idea es crear una función que funcione por **cualquier** dataframe, no sólo con lo que hemos trabajado hasta ahora.

Muestra el resultado de la función en alguno de los datasets de ejemplo que contiene el paquete seaborn. Por ejemplo, *iris*, *penguins* o *titanic*.

Ten en consideración que en el siguiente sprint trabajarás exclusivamente con gráficos. El objetivo de este ejercicio no es crear gráficos muy elaborados, sino resolver una necesidad de forma rápida y automática.

```
import pandas as pd          # manipulacion de datos
import matplotlib.pyplot as plt # graficos
import seaborn as sns        # visualizacion + datasets
import os                    # archivos y carpetas

#Crear funcion para generar graficos automaticamente.

def generar_graficos_automaticos(df, carpeta_salida="graficos"):

    # Crear carpeta donde se guardarán los gráficos, exist_ok=True evita error si la carpeta ya existe
    os.makedirs(carpeta_salida, exist_ok=True)

    # Recorrer todas las columnas del DataFrame
    for columna in df.columns:

        # Eliminar valores nulos para evitar errores en los gráficos
        serie = df[columna].dropna()

        # Si después de eliminar NaN la columna queda vacía
        # saltamos a la siguiente columna
        if serie.empty:
            continue

        # Crear una nueva figura para cada gráfico
        plt.figure(figsize=(6,4))
```

```
# hago los graficos dependiendo el tipo de la columna
# Detecta si la columna es NUMERICA y ademas se asegura de que no sea booleana(1 y 0 numericos)
if (
    pd.api.types.is_numeric_dtype(serie) and
    not pd.api.types.is_bool_dtype(serie)
):
```

```
    # Crear histograma para ver la distribución de valores
    serie.plot(kind="hist", bins=20)

    # Título del gráfico
    plt.title(f"Histograma de {columna}")
```

```
# Date and Time
```

```
elif pd.api.types.is_datetime64_any_dtype(serie):

    # Extraer el año de cada fecha
    # luego contar cuántas veces aparece cada año
    años = serie.dt.year.value_counts().sort_index()

    # Crear gráfico de barras
    años.plot(kind="bar")

    # Título
    plt.title(f"Años más frecuentes - {columna}")
```

```
# Booleanos y categoricas(lo que queda)

else:

    # Contar frecuencia de cada valor
    # mostrar solo los 10 más comunes
    serie.value_counts().head(10).plot(kind="bar")

    # Título
    plt.title(f"Top 10 valores - {columna}")

# Guardar los graficos

# Crear ruta completa del archivo
ruta = os.path.join(carpeta_salida, f"{columna}.png")

# Ajustar márgenes del gráfico
plt.tight_layout()

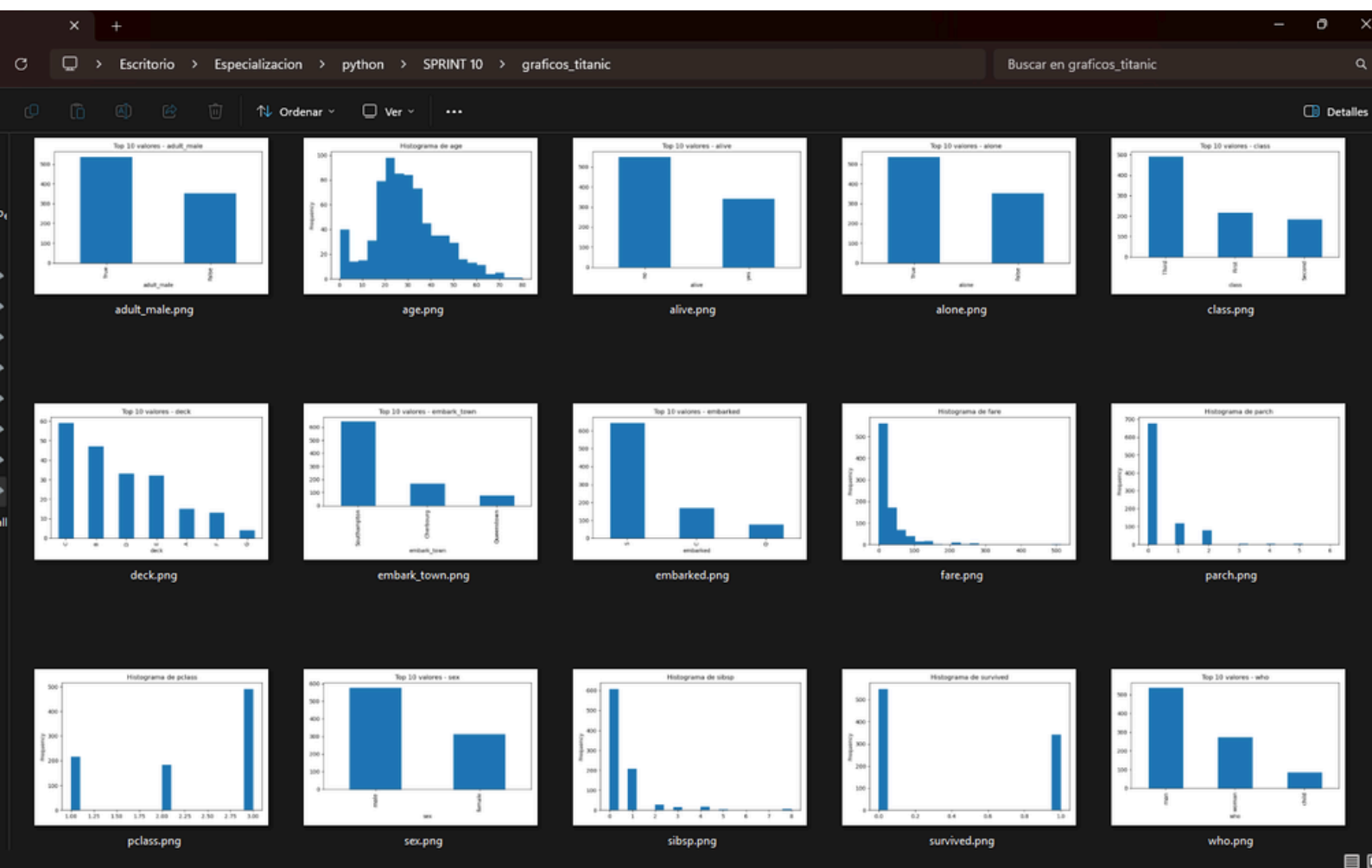
# Guardar gráfico como imagen
plt.savefig(ruta)

# Cerrar figura para liberar memoria
plt.close()
```

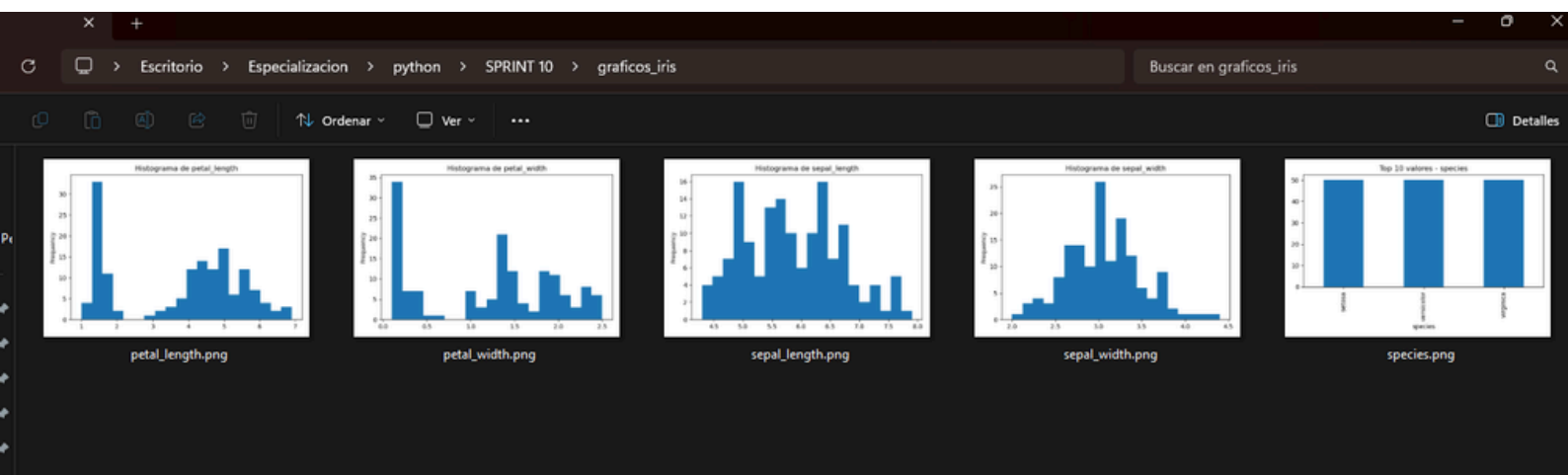
```
# Cargar datasets de ejemplo de seaborn
df_iris = sns.load_dataset("iris")
df_titanic = sns.load_dataset("titanic")
```

```
# Generar gráficos automáticos para cada dataset
generar_graficos_automaticos(df_iris, carpeta_salida="graficos_iris")
generar_graficos_automaticos(df_titanic, carpeta_salida="graficos_titanic")
```

Graficos TITANIC



Gráficos IRIS



2.

Carga el archivo `matriz_distancias.xlsx` en pandas, de modo que los nombres de filas y los nombres de columnas sean los de las ciudades. Borra "Las Palmas de Gran Canaria" y "Palma" para que podamos realizar el trayecto en coche.

Fuente: [Mejoras Rutas](#)

Nos interesa visitar todas las principales ciudades de España recorriendo la mínima distancia posible.

No hace falta que lo hagas de forma óptima, nos interesa que desarrolles una solución razonable utilizando las herramientas que tienes actualmente.

Por ejemplo, una aproximación sencilla (que no óptima) sería yendo siempre a la ciudad más cercana que no hayamos visitado todavía

Haz una función que dada la matriz de distancias y la ciudad de origen, haga una propuesta de ruta que sea lo más corta posible que puedas, devolviendo una lista con el orden de visita. Da también la distancia total recorrida.

(EXTRA) Desde qué ciudad la ruta sería más corta con el algoritmo planteado

```
import pandas as pd
# cargo el archivo de excel coloncado la primer columna como indice
df_dist = pd.read_excel(
    "matriz_distancias.xlsx",
    index_col=0
)
# borro las ciudades que no se puede acceder en coche directamente.
df_dist = df_dist.drop(
    index=["Las Palmas de Gran Canaria", "Palma"],
    columns=["Las Palmas de Gran Canaria", "Palma"]
)
#Creo la funcion para calcular la ruta, tomando la matriz y la ciudad dada
def ruta_simple(matriz, ciudad_origen):
    ciudades_no_visitadas = list(matriz.index)
    ciudades_no_visitadas.remove(ciudad_origen) #creo la variable de ciudades no visitadas eliminando la ciudad de origen para que no haya repetidos.

    # creo los valores iniciales de la ruta, la distancia total y la ciudad actual
    ruta = [ciudad_origen]
    distancia_total = 0
    ciudad_actual = ciudad_origen
```

```

while ciudades_no_visitadas:      #utilizo while ciudades_no_visitadas, para que se repita hasta que no haya mas ciudades no visitadas.

    # Buscar ciudad más cercana no visitada
    distancias = matriz.loc[ciudad_actual, ciudades_no_visitadas]
    siguiente_ciudad = distancias.idxmin()      #siguiente ciudad = nombre de la ciudad con la distancia minima

    distancia_total += distancias.min()      # sumo el valor minimo a la distancia total

    ruta.append(siguiente_ciudad)      #agrego siguiente ciudad a la ruta y la elimino de las ciudades no visitadas para que no se repita
    ciudades_no_visitadas.remove(siguiente_ciudad)

    ciudad_actual = siguiente_ciudad      #ahora la ciudad actual es la siguiente ciudad y repite el proceso.

return ruta, distancia_total

ruta, distancia = ruta_simple(df_dist, "Vigo")

print("Ruta propuesta:")
print(ruta)

print("Distancia total:")
print(distancia)

```

4) ✓ 0.0s

```

Ruta propuesta:
['Vigo', 'Gijón', 'Bilbao', 'Valladolid', 'Zaragoza', 'Valencia', 'Alicante', 'Murcia', 'Córdoba', 'Sevilla', 'Málaga', 'Hospitalet de Llobregat', 'Barcelona']
Distancia total:
2873.0

```

Creo funcion para obtener la mejor ruta y mejor ciudad de origen.

```

def mejor_origen(matriz):

    mejor_distancia = float("inf")
    mejor_ciudad = None
    mejor_ruta = None

    for ciudad in matriz.index:      #por cada ciudad en el indice de la matriz, ejecutar la funcion ruta_simple y encontrar la ciudad de origen con la ruta mas corta.

        ruta, distancia = ruta_simple(matriz, ciudad)

        if distancia < mejor_distancia:
            mejor_distancia = distancia
            mejor_ciudad = ciudad
            mejor_ruta = ruta

    return mejor_ciudad, mejor_distancia, mejor_ruta      #devolver la mejor ciudad, la mejor distancia y la mejor ruta

ciudad, distancia, ruta = mejor_origen(df_dist)

print("Mejor ciudad de inicio:", ciudad)
print("Distancia:", distancia)
print("Ruta:", ruta)

```

[35] ✓ 0.1s

```

... Mejor ciudad de inicio: Barcelona
Distancia: 2778.0
Ruta: ['Barcelona', 'Hospitalet de Llobregat', 'Zaragoza', 'Valencia', 'Alicante', 'Murcia', 'Córdoba', 'Sevilla', 'Málaga', 'Valladolid', 'Gijón', 'Bilbao', 'Vigo']

```